

Learning-Augmented Page Replacement Under a Workload Shift

Student Name — Student ID
CSE-307 Operating Systems — Section B

Abstract

This paper evaluates FIFO, LRU, Belady's optimal replacement, and a lightweight decision-tree eviction heuristic on a seeded synthetic trace. The trace begins with locality-heavy references and changes at its midpoint to broad random access. The experiment shows how increasing the working set changes fault behavior and tests whether simple history features can adapt to the shift. The optimal policy provides an offline lower bound; it is not an implementable online policy.

1. Problem framing

Virtual memory allows a process to use a logical address space larger than physical memory, but a reference to a non-resident page causes a page fault. When a frame is full, the replacement policy chooses a victim. FIFO is simple but ignores reuse, while LRU uses the recent past as a proxy for near-future reuse. Belady's optimal policy chooses the page whose next use is farthest away and is useful as an offline comparison [1,2]. The course brief asks whether a lightweight learned layer can respond when the workload changes [4].

2. Implementation and learned component

The Python program implements all four policies. The learned policy uses a depth-4 decision tree. For each resident candidate, its features are age since last reference, frequency in the previous 64 references, and whether it was touched in the previous eight references. Training uses an independent seeded trace. Labels identify the candidate with the farthest next use in a 96-reference horizon. During evaluation, only current/history features are supplied. This is an interpretable heuristic, but the labels use future information during training, so it is not a fully online production predictor [3].

3. Experimental setup

The evaluation trace contains 1,000 references and uses random seed 307. References 0–499 are locality-heavy: 82% of accesses select from a small hot set, with occasional sequential accesses. At reference 500, the distribution shifts to uniform random references over 32 pages. Each policy is tested with 4, 8, and 12 frames; the main comparison uses 8 frames. The same trace is replayed for every policy. The exact trace and metadata are stored in results/.

4. Results and analysis

Table 1 reports the 8-frame results generated from results/metrics.csv. Optimal has the fewest faults because it sees the future. LRU benefits from repeated hot-page references in the first phase, while FIFO can evict a frequently reused page simply because it arrived earlier. After the shift, the larger effective working set reduces all hit ratios. The learned policy performs well during locality, but its history features and training distribution do not generalize as well to the random phase; this illustrates that learning can adapt to patterns but can also amplify distribution mismatch.

Table 1. Eight-frame results from the seeded run.

Policy	Local faults	Local hit	Shift faults	Shift hit
FIFO	129	0.742	379	0.242
LRU	109	0.782	382	0.236
Optimal	56	0.888	234	0.532
Learned	81	0.838	396	0.208

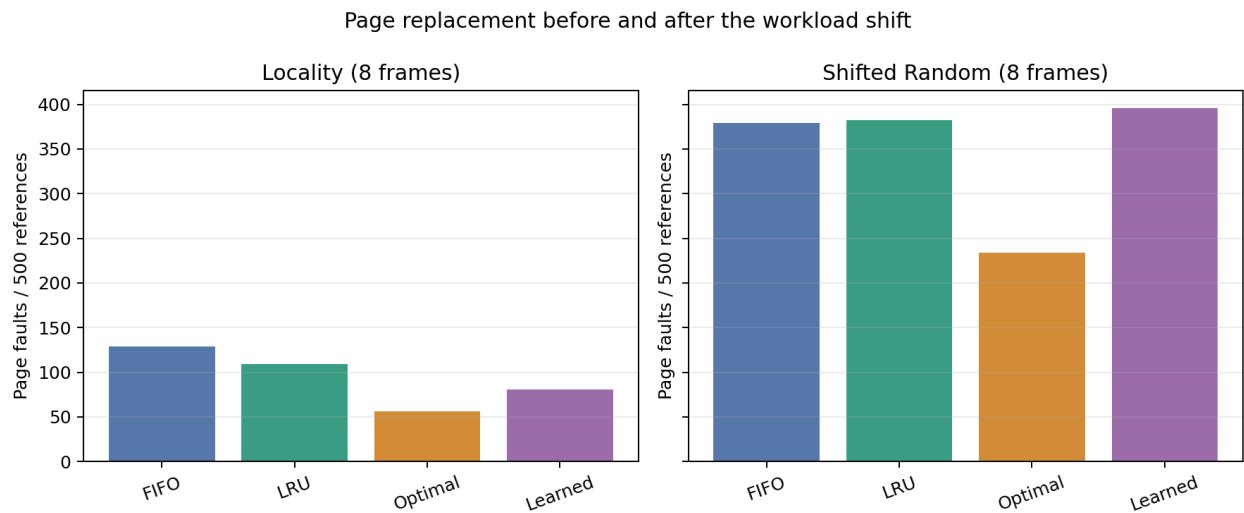


Figure 1. Fault counts before and after the distribution shift for eight frames.

5. Conclusion

The experiment connects classical memory-management policies to a small adaptive component without changing the workload between policies. The shift demonstrates why a policy tuned to stable locality may degrade when the working set broadens. Future work should use multiple seeds, online labels, confidence calibration, and real traces before making deployment claims.

References

- [1] L. A. Belady, 'A study of replacement algorithms for a virtual-storage computer,' IBM Systems Journal, 1966.
- [2] R. L. Mattson et al., 'Evaluation techniques for storage hierarchies,' IBM Systems Journal, 1970.
- [3] scikit-learn developers, DecisionTreeClassifier documentation, 2025.
- [4] CSE-307 course staff, Term Paper Brief — Section B, 2026.